ECE 51032: Computational Methods for Power System Analysis

Data-Center Connection Planning Game

Project and Student Tutorial

Purdue University, Fall 2026

Part I is the official project description, in two pages. **Part II** is the tutorial: the game, the model, the tools, and strategy, with a walk through your first plan.

Part I: The Project

The project

Your role. Your team is a large-load developer: you build data centers, and your goal is to connect gigawatts of computing load to the Texas grid as quickly and as profitably as possible. The grid is a 2,000-bus synthetic model of Texas (section 7), and the instructor plays the grid operator.

Interconnection options. Each site is one row of a CSV file (section 8):

- a bus (where the site connects) and a connection capacity C MW (the load you ask the grid to serve);
- optionally, a co-located generator of $G \leq C$ MW that is used to offset load at the location, i.e., the net load for your load and generator there is $C - G$;
- optionally, a co-located 4-hour battery of P MW and $4P$ MWh, that will be operated by the system operator according to multiperiod economic dispatch to help integrate more load into the network (section 3).

A team's sites may add up to at most x GW of connection capacity per submission, where x will be set once we know how many teams take part.

The game. You are not alone on the grid. Every team's plan is evaluated together on the same network, to ensure the feasibility of the grid during real time operation. If the plans do not all fit, the plans will be curtailed down systematically to maximize the total load interconnected (section 2.2). So the best place to build depends on where the other teams build: a bus that looks empty may be crowded once everyone picks it. After each submission you see every team's service fraction and a grid map of the total load per bus, but never another team's plan (section 14).

Teams and communication

Every team should have three students, formed by yourselves. You must form a team by **November 15th** for this project (or an alternative project); but forming the team earlier will likely be beneficial. Each team will be assigned a fixed `team_id` (chosen by yourselves) all game long, and one member uploads each file. Questions, announcements, and finding teammates go through the Discord channel (link: `https://discord.gg/5E7cSeH2jj`). Teams may talk to and coordinate with each other, but do not share your CSV or your exact plan with another team. You may use AI tools (e.g., coding agents) for any part of the work and it is encouraged to do so (section 5). Your team is responsible for every number it submits and must be able to explain its plan in the final presentation.

The three submission rounds

Every submission round plays the same game on the same data: you are developing data-center siting plans, possibly with co-located generation and storage, that will be in effect for the next ten years, made in two submissions per week (fig. 1). The practice round (Round 1) spans two calendar weeks, Nov 16–29.

Round	Dates	Submission 1 due	Submission 2 due	Grid energy cost
1 (practice)	Nov 16–29	Sun Nov 22, 11:59 PM	Sun Nov 29, 11:59 PM	none
2 (graded)	Nov 30–Dec 6	Thu Dec 3, 11:59 PM	Sun Dec 6, 11:59 PM	none
3 (graded)	Dec 7–13	Thu Dec 10, 11:59 PM	Sun Dec 13, 11:59 PM	at the LMPs

- Submission 1: connect your first sites.
- Submission 2: the Submission 1 results are released first, and every team's Submission 1 load/generation/storage becomes part of the base system. List only new sites and they compete for the room that is left.
- Everything resets between weeks, so you can use what you learned the week before.
- What you submit: one CSV per team per submission, named `team{x}_week{x}_sub{x}.csv` (e.g., `team07_week2_sub1.csv`) and uploaded to BrightSpace. Check it first with `ercot-game-check-csv` (sections 8 and 13): an invalid or late file scores 0. The free late days for the course cannot be used for the project.
- At the end: a short team presentation (last week of class) and a report on what you did, why, and optionally, some insights on explaining the results of the game.

Scoring rules

Each submission is scored in US dollars, once, for the ten-year value of the sites it builds (section 2; worked example in section 2.1): score = IT value − energy cost − generator operating cost − generator capital − storage capital − site cost.

Term	Value	How it is charged
IT value	\$1,000 per MWh	For every MWh of your load that is actually served.
Energy cost	LMP at your site	Grid purchases times the LMP. Week 3 only.
Generator operation	\$80/\$150/\$250 per MWh	First 25% / next 50% / last 25% of capacity.
Generator capital	\$2.0M per MW	Once, on the allocated generator capacity αG.
Storage capital	\$400k/MW + \$250k/MWh	Once, on αP and αB; a 4-hour battery costs \$1.4M per MW.
Site cost	\$25M per MW	Once, on the allocated connection capacity αC.

Here $\alpha \in [0, 1]$ is a site's allocated share. The representative days make up one typical year, and the ten years count in net present value at a 7% discount rate: NPV $= \sum_{y=1}^{10}$ (yearly operating net revenue)$/1.07^y -$capital, where $\sum_{y=1}^{10} 1.07^{-y} = 7.02$. So one MW served in every hour earns $8{,}760 \times \$1{,}000 \times 7.02 =$ \$61.5M, against \$25M of site cost. Capital is paid up front and is not discounted.

Grading

The project counts for 40 points of the final grade:

- 30 points: the presentation, the report, and your work.
- 10 points: your final game score S, the sum of your four Week 2 and Week 3 submission scores.
  - 5 absolute points: $5 \times \min\big(1, \max(0, S/(2B))\big)$, where $B = x \times \$49.67\text{B}$ is the final score S of a reference team that runs its full x GW off the grid ($G = C$).
  - 5 rank points: 5, 4, 3, 2, or 1 for the top 20%, 20–40%, 40–60%, 60–80%, or bottom 20% of teams.
- Bonus, up to 5 points: 3, 2, and 1 for the top three final scores, and up to 2 for reporting non-trivial and non-intended issues/bugs in the project design.

Purdue's academic integrity policy applies. To question an evaluation, email Minghao Mou (mmou@purdue.edu) within 7 days of the results release.

Resources and links

- Part II: setup (section 4), an AI-agent walkthrough (section 6), and strategy (section 11).
- Code and data: `https://github.com/MhaoMou/ercot-connection-game/`.
- Discord: `https://discord.gg/5E7cSeH2jj`.

Part II: The Tutorial

Quick start

1. Read Part I, the project (to The game).
2. Set up the GitHub repository (section 4).
3. Try the AI-agent walkthrough on the IEEE 14-bus demo (section 6).
4. Read sections 1 and 2 so you know what earns points.
5. Explore the representative data (section 7).
6. Write your plan as a CSV and run the checker on it (section 8).
7. Preview the plan, change one thing at a time, and repeat (sections 9 and 11).
8. Go through the checklist in section 13 and submit.
9. When the results come out, read them as described in section 14.

Contents

Part I: The Project 1
The project 1
Teams and communication 1
The three submission rounds 1
Scoring rules 2
Grading 2
Resources and links 2

Part II: The Tutorial 3
1 The game in one page 5
1.1 What you decide . . . 5
1.2 Schedule and data . . . 5
2 How your score is computed 7
2.1 A worked example . . . 7
2.2 You are not the only player . . . 7
3 How the grid operator runs your plan 8
3.1 Storage . . . 9
3.2 Prices . . . 9
4 Set up your computer 10
5 Using AI for this project 10
5.1 Which kind of tool . . . 11
5.2 How to write a good prompt . . . 11
5.3 Work in a loop . . . 11
5.4 Check what the AI tells you . . . 12
5.5 Privacy . . . 12
6 Walk through the workflow with an AI agent 12
6.1 Step 1: understand the project . . . 13
6.2 Step 2: look at the data . . . 13

6.3 Step 3: hosting capacity . . . 14
6.4 Step 4: write and check a plan . . . 15
6.5 Step 5: preview the plan . . . 15
6.6 Step 6: what if a rival copies you? . . . 16
7 Explore the data **16**
7.1 Hosting capacity: how much room a bus has . . . 18
8 Write and check your CSV **18**
9 Preview your plan **19**
10 Putting it together: your first plan **20**
11 Develop a strategy **21**
11.1 A two-team example . . . 21
11.2 Run controlled experiments . . . 22
12 Troubleshooting **23**
13 Before you submit **23**
14 After each submission: reading your results **24**
15 Frequently asked questions **24**
16 Glossary **26**

Appendices 27

A Hosting capacity in detail **27**
A.1 Formulation . . . 27
A.2 Computing it . . . 29
B The evaluator in full **30**
B.1 Solving a day quickly: Benders decomposition . . . 31
C The site allocation in detail **32**
C.1 What a fraction does . . . 33
C.2 The rule . . . 33
C.3 What the rule rewards . . . 33
C.4 How it is computed . . . 33
D Where the numbers come from **34**
E Working as a team **35**

This part walks you through the project from start to finish: what the game is, how your plan is scored, how to explore the data, how to write and check your submission, and how to use AI tools along the way. The numbers come from the scoring-rules files in `data/teaching/`, and `docs/HYPERPARAMETERS.md` lists every game parameter with its source.

1 The game in one page

Your team is a data-center operator. You choose where to connect computing load to a public-data-informed planning model of the Texas grid, how large (in MW) connection is, and whether to build your own generation and battery storage collocated with it. The instructor, acting as the grid operator, then runs one DCOPF dispatch with every team's plan in it, and scores each team. The grid is the public ACTIVSg2000 test system [18]: 2,000 buses, 3,206 branches, and 544 generators, stored in the MATPOWER case format [24].

1.1 What you decide

Your plan is a list of *sites*. For each site you choose four numbers:

Decision	Unit	Meaning
Bus	–	Where the site connects to the network (a `bus_id`).
Connection capacity C	MW	Your requested computing load cap at this site.
Generator capacity G	MW	Optional private generator at the site, at most C.
Storage power P / energy B	MW / MWh	Optional battery: either both zero, or $B = 4P$.

You only choose sites and four capacities. The grid operator decides how your generator and battery run in each operating period. Your generator and battery serve only your own load: they never export to the grid or to another team. The connection capacities of all your sites add up to at most x GW per submission (x is set once we know how many teams take part).

1.2 Schedule and data

The schedule and the timeline are set out in The game (fig. 1). In short: three weeks, each playing the same ten-year game in two submissions; Week 1 is practice, Weeks 2 and 3 count, and only Week 3 charges energy.

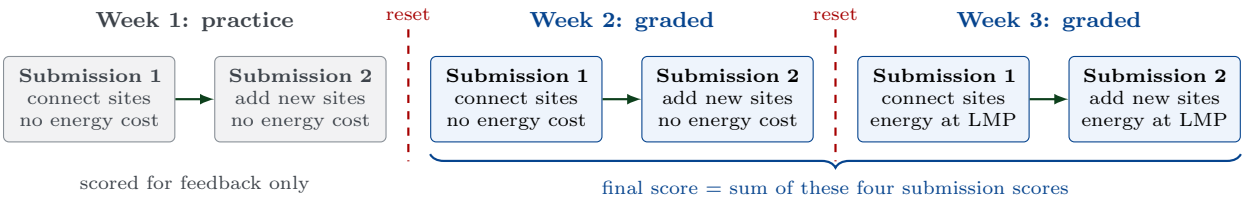


Figure 1: The submission timeline. Load connected in Submission 1 becomes base load for Submission 2 of the same week; everything resets between weeks.

Every submission uses the same representative data for the next ten years, in `data/processed/activsg2000_representative_v1/`: five typical days of 24 hourly periods, 120 periods in all. They were picked from a year of hourly load data: four days that each stand for a group of similar days, plus the peak-load day, which stands for itself.

Day	Periods	Stands for (days per year)	Daily energy (GWh)	Peak (GW)
1 (winter)	0–23	83.77	1,569	69.4
2 (spring)	24–47	99.73	1,493	66.2
3 (summer)	48–71	82.77	1,653	72.9
4 (late summer)	72–95	97.73	1,729	76.1
5 (peak-load day)	96–119	1	1,780	78.2

The days sum to 365. Your CSV uses one row per connection site; the evaluator applies that plan to all 120 periods.

From 120 hours to ten years

Period t belongs to a day that stands for d_t days of the year, so it stands for d_t hours of each year ($\sum_t d_t = 8{,}760$) and $10\,d_t$ hours of the ten years (87,600 in all). Every per-MWh amount (IT value, energy, generator fuel) earned in period t is counted in present value:

$$\text{value} = \sum_t (\$ \text{ per hour in period } t) \times d_t \times A, \qquad A = \sum_{y=1}^{10} \frac{1}{1.07^y} = 7.02,$$

where A adds up the ten years, each discounted at 7% per year. Since $A = 0.702 \times 10$, a present value is 0.702 times the plain ten-year total. For one MW served in every hour: $87{,}600 \times \$1{,}000 \times 0.702 = \$61.5\text{M}$. (The rules file stores the weights $d_t A$ for each period.) Capital is paid once, up front, and is not discounted. With `daily\_exact\_cycle`, a battery starts each representative day at $B/2$ and must end it at exactly $B/2$.

Key idea

In Submission 2, the load that all teams connected in Submission 1 is part of the base load. A site added in Submission 2 earns the same ten-year value as one built in Submission 1, but it competes only for the room that is left.

Submission 2 may find little room

Submission 1 may leave little or no room on the grid. A Submission 2 site is then scaled down, possibly to nothing, and earns little. You pay only for the allocated capacity, but a generator still costs fuel. Before you submit, recompute hosting capacity on the Submission 2 data your instructor releases (section 7.1) and preview your plan on it (section 9).

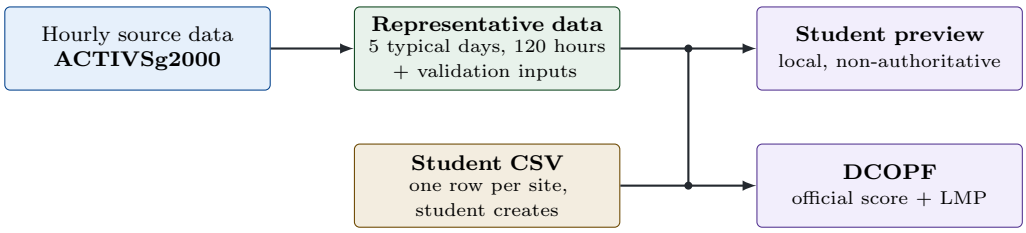


Figure 2: What you receive and what you submit. The released package gives you the network plus scenario context and validation inputs. You create the plan (CSV submission), and can preview it yourself, but only the official evaluator produces your score.

2 How your score is computed

For each submission, your score in US dollars is

$$\begin{aligned}\text{score} = \text{IT value} - \text{energy cost} - \text{generator operating cost} \\ - \text{generator capital} - \text{storage capital} - \text{site cost.}\end{aligned}$$

The coefficients are in the table in The game: IT value \$1,000/MWh served, energy at the LMP in Week 3 only, a generator curve of \$80/\$150/\$250 per MWh, all over ten years, and capital of \$25M/MW for a site, \$2.0M/MW for a generator, and \$400k/MW plus \$250k/MWh for storage, charged once when built, on the allocated capacities (αC, αG, αP, αB; section 2.2). Storage has no operating cost. This section shows what those numbers mean for a plan.

2.1 A worked example

Suppose you build one 50 MW site and nothing else, and all of its load is served in every period. Baseline LMPs are about \$18/MWh.

In Week 1 or Week 2 (no energy cost):

$$\begin{aligned}\text{IT value} &= 50 \times 87{,}600 \times 1{,}000 \times 0.702 = \$3{,}076.3\text{M} \\ \text{site cost} &= 50 \times 25\text{M} = \$1{,}250.0\text{M} \\ \text{score} &= \$1{,}826.3\text{M}\end{aligned}$$

In Week 3 (energy at the LMP):

$$\begin{aligned}\text{energy cost} &\approx 50 \times 87{,}600 \times 18 \times 0.702 = \$55.4\text{M} \\ \text{score} &\approx \$1{,}770.9\text{M}\end{aligned}$$

Each served MW earns about \$61.5M (present value) over ten years against a one-time site cost of \$25M: about \$36.5M net per MW served from the grid.

The baseline: running off the grid

A site with its own generator of the same size ($G = C$) never needs the grid, so it is always served in full. Per MW it earns \$61.5M, pays \$25M of site cost and \$2.0M of generator capital, and burns fuel in every hour: at an average of \$157.5/MWh on the \$80/\$150/\$250 curve, that is \$9.7M over ten years. It nets about \$24.8M per MW, against about \$36.5M for a MW served from the grid. A team running its full x GW off the grid scores $x \times$ \$24.84B in a submission. Summed over the four graded submissions (W2S1 $x \times$ \$24.84B, W2S2 \$0, W3S1 $x \times$ \$24.84B, W3S2 \$0, since it adds no sites in Submission 2), that is the reference score $B = x \times$ \$49.67B for the absolute points. You beat it by getting load onto the grid where the grid has room.

Try it

A team adds a 30 MW site in Submission 2 of Week 3, and it is fully served at an LMP of about \$18/MWh. Estimate its score. *(Answer:* $30 \times 87{,}600 \times (1{,}000 - 18) \times 0.702 - 750M \approx \$1{,}063M$*.)*

2.2 You are not the only player

The worked example in Section 2.1 quietly assumed that all of your load is served. Whether it is depends on more than your own plan. Every team reads the same rules and does the same arithmetic, so every team is tempted by the same plan: build big, build early, and pick buses (sites) where the network looks strong.

Each of those plans can be fine on its own, and your preview, which runs without the other teams, will say so. But the spare capacity of the grid is shared. If too many teams crowd the same part of the network, the combined plan has no feasible dispatch, even though no single plan caused the problem.

> *"The hard thing in life is getting you to come out of your own selves a bit, realizing it's 'not all about you.'"*

When the combined plan of all teams is infeasible, the evaluator has to decide who gets served. It allocates the sites:

1. Site fractions. Every site i (of any team) gets its own fraction $\alpha_i \in [0, 1]$. It scales the site's connection capacity, generator and battery together: the site is treated as if you had submitted $\alpha_i C_i$, $\alpha_i G_i$, $\alpha_i P_i$ and $\alpha_i B_i$.
2. Serve the most. The fractions maximize the total served capacity $\sum_i \alpha_i C_i$ over all sites, subject to the joint dispatch being feasible in all 120 periods.
3. Break ties evenly. Among the fractions that serve the most, the evaluator picks the one closest to serving everyone in full (it minimizes $\sum_i C_i(1 - \alpha_i)^2$). This choice is unique, so the result does not depend on the solver.
4. Pay for what you get. Capital is charged on the allocated capacities. There is no minimum allocation: a small fraction is still served.

This changes the question you should ask. Your served load depends on what everyone else builds. The rule serves the *most megawatts*, so it favors sites that need the least of the scarce network per MW served: a site whose generator or battery covers its own peak hours, or a site at a bus where a MW of load loads the bottleneck lines less. A site that competes for the same lines with such sites can be cut a lot. So the question is not only "how much load can my site take?" but also "how much room will be left for me once the other teams have built, and what do they expect me to do?"

Section 11.1 works through a stylized two-team example at two neighboring buses.

Think about it

Suppose you expect most teams to crowd the best-connected buses. What do you gain and what do you risk by choosing a less popular location instead? What would the Week 1 results tell you about how other teams actually behave?

3 How the grid operator runs your plan

The evaluator solves a DCOPF on the ACTIVSg2000 network for each of the 120 representative hours, with all teams' sites added. The existing network, generators, and base loads come from the representative data package (in Submission 2, the base load also contains the Submission 1 sites). Figure 3 shows what one row of your CSV adds at a bus k.

For a site at bus k in period t, the evaluator adds:

- a fixed load αC (normally $\alpha \in [0, 1]$, cf. Section 2.2);
- a private generator $0 \le g_t \le G$ with the piecewise cost in The game;
- a battery with signed power p_t (positive = discharge), limited by $-P \le p_t \le P$.

Your grid purchase rate in period t is $\alpha C - g_t - p_t$. The generator and battery can only cover your own team's load at that bus; they cannot push power back into the grid.

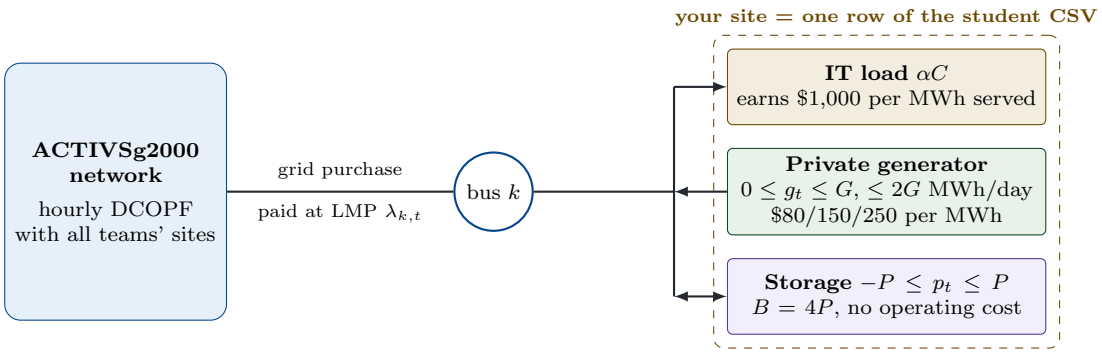


Figure 3: One site in the dispatch. The site draws αC of IT load; your generator and battery can cover part of it, and the rest is bought from the grid at the bus LMP.

3.1 Storage

Each battery follows the model below, with hourly steps and 100% efficiency. The signed variable $p_{d,h}$ is a power rate in MW: positive means discharge and negative means charge. Let $\Delta t = 1$ hour.

$$e_{d,h} = e_{d,h-1} - \Delta t\, p_{d,h}, \qquad 0 \le e_{d,h} \le B, \qquad -P \le p_{d,h} \le P, \qquad \Delta t\, p_{d,h} \le e_{d,h-1},$$
$$e_{d,0} = 0.5B, \qquad e_{d,24} = 0.5B \qquad \text{for each representative day } d.$$

The battery starts every representative day half full and must end it at exactly half full after 24 hours (fig. 4). There is no state-of-charge carryover from one day to the next. Discharge at a bus cannot exceed your own load there.

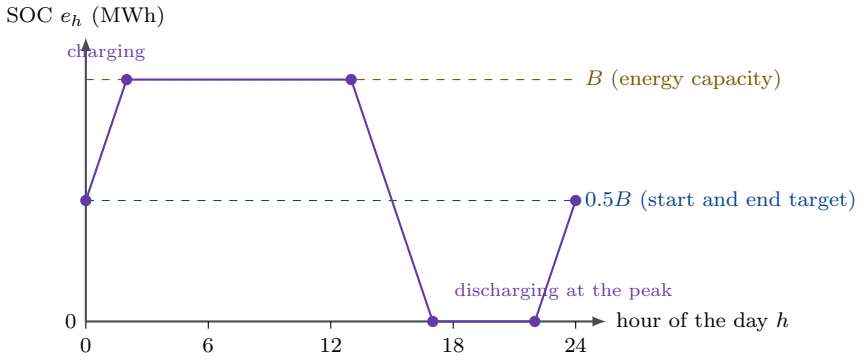


Figure 4: An illustrative one-day battery schedule with $B = 4P$: it starts at $0.5B$, stays between 0 and B, moves by at most P MWh per hour, and returns to exactly $0.5B$ at the end of the day. The evaluator chooses this schedule independently for each representative day.

3.2 Prices

The LMP at your bus is the dual of that bus's power balance in the joint dispatch, clipped at zero. In Week 3, energy is settled against LMPs computed from the actual joint dispatch containing all accepted teams. Optional baseline LMP artifacts may be generated for diagnostics. With teams' load added, lines can congest and the LMP at your bus can rise.

Try it

A battery has $P = 5\,\text{MW}$ and $B = 20\,\text{MWh}$. It starts a representative day at the required state of charge. Can it discharge at 5 MW for two hours in a row? What is the state of charge after each hour? *(Answer: it starts at 10* MWh*, is at 5* MWh *after one hour and 0* MWh *after two; it cannot discharge in a third hour until it has charged. It must also return to exactly 10* MWh *by the end of the day.)*

4 Set up your computer

You need Python 3.11 or newer and the course repository. From a terminal:

```
git clone https://github.com/MhaoMou/ercot-connection-game.git ercot-game
cd ercot-game
python3 -m venv .venv
source .venv/bin/activate        # Windows: .venv\Scripts\activate
python -m pip install -e ".[tutorial]" pandas
```

Run every command in this tutorial from the repository root with the virtual environment active. Check the installation by validating the example submission (section 8 explains the output):

```
ercot-game-check-csv --file examples/eight_team_round/submissions/team-01.csv
```

The files you will use most:

Path	What it is
`docs/HYPERPARAMETERS.md`	Every game parameter, with its value and source.
`data/processed/activsg2000_representative_v1/`	Representative data: five typical days, 120 hourly periods.
`data/teaching/dispatch_scoring_rules_representative_v1.json`	Scoring coefficients and the ten-year weights used by the preview and the evaluator.
`ercot-game-check-csv`	Checker for your CSV (also `scripts/check_submission.py`).
`ercot-game-preview`	Approximate scoring run for your plan.
`docs/student_tutorial/notebooks/`	Small companion notebooks (see below).

The notebooks in `docs/student_tutorial/notebooks/` use tiny made-up fixtures, not the real data. They are useful for seeing the mechanics in isolation: `03_three_bus_dcopf.ipynb` solves a three-bus DCOPF with the same solver the evaluator uses, `04_storage_soc.ipynb` traces a battery's state of charge, and `05_csv_validation.ipynb` calls the checker from Python.

5 Using AI for this project

You are encouraged to use AI tools such as ChatGPT, Claude, Gemini, GitHub Copilot, Claude Code, or Codex. You are still responsible for every number you submit and must be able to explain your plan. This section is for anyone who has not used these tools much. If you are already familiar with AI tools, you can skip this section.

5.1 Which kind of tool

- **Chat assistants** (ChatGPT, Claude, Gemini in a browser) are best for explaining concepts, writing short scripts, and decoding error messages. They only see what you paste in. Most of them can be used for free. However, without subscriptions, we may be limited to only the use of basic models.
- **Coding assistants/AI agents** [recommended] (Claude Code, Codex, Copilot in your editor) can read the GitHub repository and run commands for you. They are faster once you are comfortable, but check what they change. Most of them require subscriptions. All the demos in the tutorials used coding assistants, since it is not easy for char assistants to manipulate local files.[1]

If you are new to this, start with a chat assistant and copy code into your own files. Each tool has an official getting-started guide:

Tool	Kind	Getting started
ChatGPT	Chat	What is ChatGPT: FAQ
Claude	Chat	Get started with Claude
Gemini	Chat	Use Gemini Apps
GitHub Copilot	Coding (editor)	Quickstart for GitHub Copilot in your IDE
Claude Code	Coding (terminal)	Claude Code quickstart
Codex	Coding (terminal)	Codex CLI

GitHub Copilot is free for verified students, see Access GitHub Copilot for free as a student.

5.2 How to write a good prompt

The AI knows DCOPF but knows nothing about this project. Give it four things:

- the **goal**,
- the **context** (paste file headers, a few rows, error messages),
- the **constraints**,
- the **form** of answer you want.

Weak prompt

Write code to analyze the data.

Better prompt

I am working in Python with pandas. I have a file `branch_limits.csv` with columns `branch_id, from_bus_id, to_bus_id, reactance_pu, thermal_limit_mva, status`. Here are its first three rows: [paste]. Write code that, for each bus, adds up `thermal_limit_mva` over in-service branches (`status == 1`) connected to that bus, and prints the 20 buses with the largest total. Explain each line briefly.

5.3 Work in a loop

Treat the AI as a pair programmer. Ask for something small, run it, and paste back exactly what happened, including the full error message. Repeat until it works, then ask it to explain the final version so you understand it.

[1] There are in fact some advanced tools that allow chat assistants to get access to and manipulate local files. You can search for those tools online. However, using those tools comes with risks of getting your account suspended.

Ask AI

I ran your script and got this error: `KeyError: 'thermal_limit'`. The column names are exactly: `branch_id,from_bus_id,to_bus_id,reactance_pu,thermal_limit_mva,status`. Please fix the script.

Example AI reply

The column is called `thermal_limit_mva`, not `thermal_limit`. Change line 6 to `branches["thermal_limit_mva"]` and run it again.

5.4 Check what the AI tells you

AI tools are often confidently wrong about project-specific details (and this is the reason why you need to provide enough context to AI agents). They might invent a CSV column, assume you schedule the battery yourself, or use made-up prices. Before you rely on an answer:

- **Run it.** Code that has not run is a guess.
- **Check the numbers** against this tutorial and the rules JSON.
- **Try a case you can do by hand**, such as the worked example in section 2.
- **Ask for its assumptions**: "What did you assume that is not in the files I gave you?"

5.5 Privacy

Do not upload passwords, access tokens, or other credentials. Your plan is your team's competitive information, so do not share it with other teams,[2] including through shared chat links. Public data from the repository is fine to paste.

6 Walk through the workflow with an AI agent

This section shows the whole workflow, from reading the rules to a joint evaluation with a rival team, done by talking to an AI agent. It uses a small IEEE 14-bus demo that ships with the repository (`data/teaching/ieee14_demo_representative`): 14 buses, 20 lines, 5 generators, and the same five representative days (120 hourly periods) and ten-year scoring as the game. Every command runs in a few seconds, so you can repeat the session yourself. The demo is for practice only: the game itself runs on `ACTIVSg2000`, and the demo's numbers do not carry over.

How the session was run. A coding agent (Claude Opus 5.5) was started in a fresh clone of the repository with the package installed (section 4). It had not seen the repository before. The six prompts below were typed one after another in the same session. The replies are real but shortened, with `[...]` marks a cut. The complete prompts and replies are in `docs/student_tutorial/ai_walkthrough/`. To start an agent yourself, open a terminal in the repository folder and run, for example, `claude` (Claude Code) or `codex` (Codex CLI).

Watch how each prompt is written. It states the goal, names the files and commands, sets constraints ("do not change any files", "under 150 words"), and asks for the form of the answer ("a table", "show the commands"). After every reply, a box lists what you should check yourself before you rely on it.

[2] However, it is possible to cooperate with other teams. It is also an interesting direction to explore.

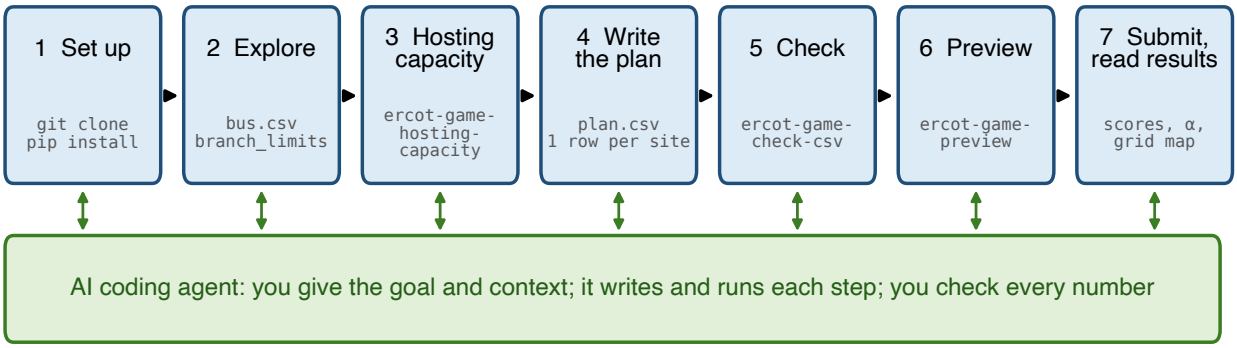


Figure 5: The workflow in the repository, with an AI agent in the loop. Each step has its own section: set up (section 4), explore (section 7), hosting capacity (section 7.1), write and check the plan (section 8), preview (section 9), and read the results (section 14). The six prompts below walk through steps 2–7.

6.1 Step 1: understand the project

Prompt 1

I just cloned this repository for my ECE 51032 course project. Read the README and Part I of `docs/student_tutorial/main.tex` (context), then explain in under 150 words (constraint): what my team submits, how it is scored, and which commands I will use. (deliverables) Do not change any files (constraint).

Agent reply (shortened)

What you submit: one CSV per submission, two a week for three weeks (six files in all). [...] The rules are $G \leq C$ and storage either zero or $B = 4P$. Submission 2 lists only new sites. You also give a final presentation and a report.
Scoring: score = IT value ($1,000/MWh served, over 10 years) − energy cost (charged at the LMP, Week 3 only) − generator operating cost − capital costs [...]. Your final score is the sum of the four Week 2 and 3 submissions. [...]
Commands: `ercot-game-check-csv` [...] and `ercot-game-preview`. Note: the README describes an older six-hour setup ($6,500/MWh). Follow Part I.

Check it yourself

When this session was recorded, the README still described an earlier version of the game; it has since been updated. The agent noticed that two documents disagreed and said which one it trusted, which is exactly what you want. But it left out a tool you will need, `ercot-game-hosting-capacity`. Compare its summary with The game and The game rather than trusting a summary.

6.2 Step 2: look at the data

Prompt 2

We will practice on the small IEEE 14-bus demo package in `data/teaching/ieee14_demo_representative` before touching the real grid. Summarize it: number of buses, lines, and generators, total load, the number of operating periods, and the five lines with the smallest thermal limits. Show the commands you ran.

Agent reply (shortened)

Buses: 14. **Lines:** 20, all in service. **Generators:** 5, with 772.4 MW total capacity [...]. **Load:** the base load is 259.0 MW. Across the periods it ranges from 224.5 to 301.7 MW [...]. **Operating periods:** 120. That is 5 representative days of 24 one-hour periods. The day weights add up to 365, and the peak day has a weight of 1. **Five smallest thermal limits:** 12–13 (20 MVA), 10–11 (25), 13–14 (25), 6–11 (30), 6–12 (30). br16 (9–10) is also at 30 MVA, so the 4th and 5th places are a three-way tie at that limit.

Check it yourself

Open `branch_limits.csv`: four lines are at 30 MVA, not three. The agent missed 9–14, which matters later because it is one of the lines that limits buses 13 and 14. Small slips like this are common; spot-check any table an agent gives you.

6.3 Step 3: hosting capacity

Prompt 3

Use `ercot-game-hosting-capacity` to compute the hosting capacity of every bus in the demo package over all 120 hours. Give me a table of each bus's minimum hosting capacity, sorted from largest to smallest, and say in one sentence what this number means. Which two buses would you shortlist for a data center, and why?

Agent reply (shortened)

```
ercot-game-hosting-capacity \
  --artifact-dir data/teaching/ieee14_demo_representative \
  --horizon 120 --period-count 120 --output-dir <tmp>/hc14
# backend highs, bus_count 14, completed_point_count 1680, failed_point_count 0, wall_time
    ↪ 1.6 s
```

The columns are in the order of `bus_ids` in the metadata file (`1,10,11,...`), not numeric order, so I matched them up by that list.

Bus	1	2	5	4	3	7	8	6	9	13	11	10	12	14
Min HC (MW)	471	429	311	241	180	178	178	169	103	51	45	45	43	38

What it means: a bus's minimum hosting capacity is the most new load you could add at that bus alone [...] that the grid can still serve in every one of the 120 hours [...]. For every bus the lowest value falls in hour 109. **Shortlist: buses 1 and 2.** [...] **One caution:** these numbers are for one bus at a time, and they don't add up. Every team will see the same table, so buses 1 and 2 may get crowded.

Check it yourself

Re-run one bus yourself (add `--bus-id14`) and compare. The agent caught a real trap, the column order of the output file. Its caveat, that every team sees the same table, is the most important line of the reply.

6.4 Step 4: write and check a plan

Prompt 4

Write a plan for team-demo with one site at each of the two buses you shortlisted, each at about half of its hosting capacity, with no generator and no battery. Save it as `runs/demo/plan.csv` and check it with `ercot-game-check-csv` against the demo package. Then make a copy with a mistake, a generator larger than its connection, and show me what the checker says.

Agent reply (shortened)

I sized each site at about half of its bus's minimum hosting capacity: 235 MW at bus 1 and 214 MW at bus 2, with no generator or battery.

```
VALID: runs/demo/plan.csv
team=team-demo sites=2 rows=2 buses=2 generators=0 storage=0 horizon=None

INVALID: runs/demo/plan_bad.csv
- row 2: generator capacity cannot exceed connection capacity
```

A generator can be at most as large as its site's connection capacity [...]. That is the $G \leq C$ rule from Part I.

Check it yourself

The agent saved the broken copy *next to* the real plan, in `runs/demo/`. The preview runs every CSV in a folder together, so keep each candidate in its own folder (the agent fixed this itself in the next step).

6.5 Step 5: preview the plan

Prompt 5

Preview the valid plan for Week 3, when energy is charged at the LMP (use `--week3`). Report the share of load served, the score, and the term that dominates it. Is the plan above break-even?

Agent reply (shortened)

The preview needs a folder of submissions, and `runs/demo/` also holds `plan_bad.csv`. So I copied the valid plan into its own folder. [...]

Term	$
IT value	+27.63 B
Energy cost (LMP)	−0.83 B
Site cost (449 MW × $25M)	−11.23 B
Score	**+15.57 B**

Share of load served: 100%. [...] **Break-even:** [...] That puts break-even at about 44% service, and the plan is far above it.

Check it yourself

Redo the arithmetic: $449 \times 87{,}600 \times \$1{,}000 \times 0.702 = \$27.6\text{B}$, as reported. But the break-even is wrong: with \$1.85M of energy per MW it is $25/(61.5 - 1.85) \approx 42\%$, not 44%. Always redo an agent's own back-of-envelope numbers.

6.6 Step 6: what if a rival copies you?

Prompt 6

Other teams will read the same hosting-capacity table. Simulate a rival, team-rival, that copies our plan exactly (same buses, same sizes), and run the joint evaluation with `ercot-game-batch` on both plans for Week 3. What service fraction does each team get, and what does that mean for our plan? Finally, list every assumption you made in this session that I should check myself.

Agent reply (shortened)

```
ercot-game-batch run --submissions-dir runs/demo/joint \
  --artifact-dir data/teaching/ieee14_demo_representative \
  --rules data/teaching/dispatch_scoring_rules_ieee14_demo_representative_v1.json \
  --week 3 --output-dir runs/demo/joint_week3
# summary: 2 teams scored, 0 rejected, scope=joint_projection
```

Together the two teams ask for 898 MW, and the grid cannot carry that, so the evaluator allocated the sites: each team is served 235.3 MW, a service fraction of **52.4%**, and scores **$8.16B** instead of $15.6B.

- Site cost is charged on the 235.3 MW we are allocated, not on the 449 MW we requested.
- A third team copying the same table would cut each of us to about 35% [...], for a score of about $5.4B.
- Smaller sites, or a spread to buses 4 and 5, are much more robust to copying.

Assumptions to check yourself: [...] I mapped the columns of `hosting_capacity_mw.npy` to buses using [...] its metadata file. A wrong mapping would change the whole table and the shortlist. [...] Both runs used the default submission number. [...] The "about 44%" figure is my own back-of-envelope arithmetic, not evaluator output.

Check it yourself

Open `runs/demo/joint_week3/evaluation.json` and find the service fractions. Asking an agent to list its own assumptions is one of the most useful prompts you can write: here it flagged both the column mapping and the break-even number it had computed by hand, which turned out to be off.

Key takeaways of this session

- The agent ran every step of the workflow on its own in a few minutes, and it read the files and `--help` before acting.
- A plan that looks safe on a single-bus table can lose most of its value once someone else builds: hosting capacities do not add, and the most obvious buses attract everyone (section 2.2).
- You remain responsible for every number. Re-run one command, redo one calculation, and open one output file for every answer you use.

7 Explore the data

The representative data is in one folder, `data/processed/activsg2000_representative_v1/`. The files you are most likely to need:

File	Contents
`bus.csv`	All valid `bus_id` values.
`demand_bus.csv`	Base load at every bus in each of the 120 hourly periods.
`representative_days.csv`	The five days: date, weight (days per year), first period index, daily energy, and peak load.
`generator_catalog.csv`	Existing generators: bus, `p_min_mw`, `p_max_mw`, `marginal_cost_c1_per_mwh`, status.
`branch_limits.csv`	Lines: `from_bus_id`, `to_bus_id`, `reactance_pu`, `thermal_limit_mva`, status.
`manifest.json`	Horizon, day weights, and validation status.

This script loads the data and prints a summary:

```
from pathlib import Path
import pandas as pd

data = Path("data/processed/activsg2000_representative_v1")
buses = pd.read_csv(data / "bus.csv")
gens = pd.read_csv(data / "generator_catalog.csv")
branches = pd.read_csv(data / "branch_limits.csv")
days = pd.read_csv(data / "representative_days.csv")
demand = pd.read_csv(data / "demand_bus.csv", index_col=0)  # 120 periods x buses

print(len(buses), "buses,", len(branches), "branches,", len(gens), "generators")
print("generator capacity", round(gens.p_max_mw.sum()), "MW")
system = demand.sum(axis=1)
print("system load", round(system.min()), "to", round(system.max()), "MW; peak in period", system.
    ↪ idxmax())
print(days[["date", "weight"]].round(2).to_string(index=False))
```

Output:

```
2000 buses, 3206 branches, 544 generators
generator capacity 96292 MW
system load 58167 to 78177 MW; peak in period 109
      date  weight
2028-01-03   83.77
2028-05-10   99.73
2028-07-12   82.77
2028-09-10   97.73
2028-09-15    1.00
```

Period 109 is 1 p.m. on the peak-load day.

Without teams' plans the LMP is nearly the same everywhere, about $18/MWh: the network is not congested. So baseline prices will not tell you where to build. What matters is where the network still has "room" once you add tens of MW. Good places to start looking are buses with strong line connections, nearby generation with spare capacity, and little base load.

Ask AI

I have these pandas DataFrames from a power-system dataset: [paste the column names of `bus_load_base.csv`, `generator_catalog.csv`, and `branch_limits.csv`]. For each bus, compute
(1) the total thermal limit of in-service branches connected to it,
(2) the total `p_max_mw` of in-service generators at that bus and at its direct neighbors, and
(3) its base load. Rank buses by a simple score of your choice, explain the score, and show the top 20.

7.1 Hosting capacity: how much room a bus has

The "room" has a name. The hosting capacity of a bus is the largest additional load that can be connected there without making the grid fail, that is, without making the DCOPF infeasible, while everything else in the network stays the same [4, 23]. Because your site draws the same load in every period, what matters is the *smallest* hosting capacity over the 120 periods, which usually occurs in the hours with the highest system load.

Here are hosting capacities for a few buses, computed with the code in appendix A for period 0 and for period 109, the hour with the highest system load:

Bus	Period 0	Period 109 (peak)
1001	391 MW	332 MW
1002	129 MW	123 MW
7419	687 MW	248 MW

For all 2,000 buses, the smallest hosting capacity over the 120 periods has a median of 371 MW; one bus in ten is below 130 MW, and 146 buses can take more than 2,000 MW. Computing all $2{,}000 \times 120 = 240{,}000$ hosting capacities takes about 20 minutes on a 32-core server with Gurobi and roughly three hours on one laptop core with HiGHS, so screen first: compute the peak hours for all buses, then all 120 hours for a shortlist.

Hosting capacity tells you where you can build and roughly how much. Use it as a screening calculation only: the official result also includes other teams.

Watch out

Hosting capacity is computed as if your site were the only new load. Other teams' sites use up the same headroom. At the peak hour, bus 1002 can host 123 MW on its own, but only about 81 MW if another team connects 60 MW at the neighboring bus 1010. The hosting capacities of different buses do not add up, and the busier a part of the network becomes, the less room each site there has (section 2.2).

Appendix A gives the optimization behind hosting capacity, how to compute it with the GitHub repository's own DCOPF solver, and how private generation and storage change it.

8 Write and check your CSV

Your submission is one UTF-8 CSV file with exactly this header and one row per site:

```
team_id,site_id,bus_id,connection_capacity_mw,generator_capacity_mw,storage_power_mw,storage_energy_mwh
```

Column	Rule
`team_id`	Your team ID. The same on every row.
`site_id`	Your own label for the site, unique within the file.
`bus_id`	Must exist in the block's `bus.csv`.
`connection_capacity_mw`	$C \geq 0$, in MW.
`generator_capacity_mw`	$0 \leq G \leq C$, in MW. All your sites' C add up to at most x GW.
`storage_power_mw`	$P \geq 0$, in MW.
`storage_energy_mwh`	$B = 4P$, in MWh (both zero means no battery).

IDs may contain letters, digits, and `_ . : -`, with no spaces and at most 128 characters. The file contains capacities only: do not add operating-period rows or any other columns.

A valid two-site example:

```
team_id,site_id,bus_id,connection_capacity_mw,generator_capacity_mw,storage_power_mw,storage_energy_mwh
team-blue,site-north,1001,40,20,10,40
team-blue,site-south,1002,15,0,0,0
```

Run the checker on it. Pass `--artifact-dir` so that it also checks your bus IDs against the representative data:

```
ercot-game-check-csv --file team-blue.csv \
  --artifact-dir data/processed/activsg2000_representative_v1
```

```
VALID: team-blue.csv
team=team-blue sites=2 rows=2 buses=2 generators=1 storage=1 horizon=None
```

The exit status is 0 for a valid file, 1 for an invalid file, and 2 for a problem with the command itself. Here is an invalid file and what the checker says about it:

```
team_id,site_id,bus_id,connection_capacity_mw,generator_capacity_mw,storage_power_mw,storage_energy_mwh
team-blue,site-north,1001,-5,20,10,30
team-green,site-south,1002,15,0,0,0
team-blue,site-north,9999999,10,0,0,0
```

```
INVALID: bad.csv
- row 2: connection_capacity_mw must be non-negative
- row 2: storage energy must equal four times storage power
- row 3: one CSV may contain only one team_id
- row 4: unknown scenario bus 9999999
- row 4: duplicate site_id site-north
```

Row numbers count the header as row 1.

In Submission 2, list only new sites. Your Submission 1 sites are already part of the base load. A Submission 2 file that reuses one of your Submission 1 `site_id`s is rejected as a whole.

Ask AI

Write a Python function `write_plan(team_id, sites, path)` where `sites` is a list of dicts with keys `site_id`, `bus_id`, `connection_mw`, `generator_mw`, and `storage_mw`. It should write a CSV with exactly this header: [paste the header], set `storage_energy_mwh` to 4 times `storage_mw`, and then run `ercot-game-check-csv --file <path>` with `subprocess` and print the output.

9 Preview your plan

The preview runs an approximate version of the evaluator on your plan alone,[3] so you can compare ideas before submitting. Put your CSV in its own folder and run:

```
mkdir -p runs/candidate-a
cp team-blue.csv runs/candidate-a/
ercot-game-preview \
  --submissions-dir runs/candidate-a \
  --week 3 \
  --output runs/candidate-a.json
```

The preview uses the representative data and the ten-year rules by default. `--week3` charges energy at the LMP; use `--week1` or `--week2` to leave it out. For Submission 2, add `--artifact-dir` with the Submission

[3] It is an approximate version because running the full version is computationally extensive. If you have enough computational power, you can ask your AI agent to write a full evaluator script and simulate other teams' plans to find what is the best you can do.

2 data your instructor releases after Submission 1: the representative data with every team's Submission 1 load already in the base load. (To try your own Submission 1 result instead, pass `--base-load-from` with a Submission 1 output folder; a Submission 2 preview without either is refused.) Every CSV in the folder is dispatched together, so keep one candidate per folder.

The preview differs from the official evaluator in two ways:

	Preview	Official evaluator
Operating periods solved	All 120 representative hours	All 120 representative hours
Storage	Not dispatched (idle), but its capital is charged	Dispatched within each representative day; half full at the start and end of each day
Other teams	Only the CSVs you put in the folder; if they exceed the grid, the same site allocation as the evaluator	All teams, with the site allocation if needed

So the preview usually understates the value of storage and overstates how much room the network has, because other teams are missing. Treat it as a way to rank your own candidates, not as a forecast of your score.

The output JSON has `"mode": "student_preview"` and `"exact": false`, plus one entry per team in `team_scores` with `score`, `served_load_mwh`, `it_value`, `energy_cost`, `generator_operating_cost`, `generator_capital_cost`, `storage_capital_cost`, `site_cost`, `feasible`, `service_fraction`, and `dropped`. If your plan does not fit the grid in some hour, the preview scales it back with the evaluator's site allocation, prints a NOTE, and records the details under `projection`. `served_load_mwh` is per represented year: compare it with $C \times 8{,}760$ to see how much of your load was served.

Watch out

The preview refuses to run on a scenario block whose `manifest.json` does not say `"validation_status": "passed"`. Your instructor releases blocks as they are validated.

10 Putting it together: your first plan

This section strings the previous steps together for Week 2, Submission 1. Keep each candidate in its own folder under `runs/` and log every run (appendix E).

1. Shortlist buses. Screen buses with the script in section 7, then compute hosting capacity for the shortlist over the peak-load hours (appendix A). Keep buses whose smallest hosting capacity is well above the size you have in mind.
2. Choose sizes. Start without a generator or battery. Keep each site well below its hosting capacity, and remember that other teams may be looking at the same buses.
3. Write the CSV by hand or with the `write_plan` helper from section 8, for example `runs/plan-a/plan.csv`.
4. Check it:

```
ercot-game-check-csv --file runs/plan-a/plan.csv \
  --artifact-dir data/processed/activsg2000_representative_v1
```

5. Preview it with Week 2 rules, so that there is no energy cost:

```
ercot-game-preview --submissions-dir runs/plan-a --week 2 \
  --output runs/plan-a.json
```

6. Read the result. Divide `served_load_mwh` by $C \times 8{,}760$ to get the share of your load that was served, and look at which cost term dominates `score`.
7. Change one thing (a bus, a size, or adding a generator), preview again, and compare (section 11).
8. Submit the best candidate, then read your results (section 14) before planning Submission 2.

11 Develop a strategy

There is no single right answer, but the scoring rules point to a few questions worth answering with data.

When to build? A site earns the same ten-year value in either submission, but Submission 1 sites are connected first: in Submission 2 they are part of the base load, and new sites compete only for the room that is left. Submission 2 is your chance to add sites once you have seen where everyone else built.

How big? A MW served from the grid earns about $36.5M over ten years after site cost; a MW run off the grid on your own generator about $24.8M (section 2). You pay only for the capacity you are allocated, and your sites add up to at most x GW. Grow grid-served capacity in steps and watch where served load stops keeping up.

Where? Since baseline prices are flat, location matters through network headroom. Shortlist buses with a large hosting capacity (section 7.1), keep your request comfortably below it to leave room for other teams, and preview the same capacity at several of them.

Private generation? Your generator may run in every hour and be as large as your site, so it is both backup for the tight hours and, with $G = C$, a way to run entirely off the grid. At $80/MWh or more it costs more than baseline grid energy (about $18/MWh), so the grid operator runs it only when the grid cannot serve you or is more expensive; in uncongested hours it sits idle and its $2.0M/MW capital buys insurance. A generator also makes your site need less of the network per MW served, which the allocation rewards (section 2.2). Look at how your bus's hosting capacity varies over the day before you size one (section 7.1).

Storage? With nearly flat prices, there is little to gain from buying low and selling high. Like a generator, a battery is mainly useful at a location where the network is tight in some operating periods. Remember that the preview does not dispatch storage, so it will not show this benefit.

Other teams. Your preview runs without them, but the official dispatch includes everyone. If the combined plan is infeasible, the allocation in section 2.2 serves the most megawatts in total, so sites that compete for the same lines as more network-efficient sites are cut first.

Use Week 1. Week 1 does not count. Use it to see how your plan is treated in the joint evaluation with everyone else.

11.1 A two-team example

Here is the game in miniature. Team A builds at bus 1002 and team B at the neighboring bus 1010, and each chooses between a small (50 MW) and a large (100 MW) site (only two options for simplicity). On its own, either bus could take 100 MW: at the peak hour their hosting capacities are 123 and 135 MW. Together they cannot. Both teams build in Submission 1, and the evaluator scores them on all 120 representative hours. Each served MW is worth about $61.5M over ten years, and each built MW costs $25M.

Payoffs, A / B ($M)	B small (50 MW)	B large (100 MW)
Weeks 1–2 (no energy cost)		
A small (50 MW)	1,826 / 1,826	1,826 / 3,255
A large (100 MW)	3,653 / 1,564	3,653 / 1,564
Week 3 (energy at LMP)		
A small (50 MW)	1,772 / 1,772	1,336 / 2,320
A large (100 MW)	2,673 / 1,115	2,673 / 1,115

- Both small. The grid can serve both sites in full.
- B large, A small. The combined plan is infeasible. The most megawatts are served by keeping A's 50 MW and giving B what is left, 89.1 MW. B pays only for those 89.1 MW.
- A large. At the shared bottleneck a MW at bus 1002 uses a little less of the line than a MW at 1010, so serving A first serves the most in total: A gets its full 100 MW and B gets 42.8 MW, whether B asked for 50 or 100. B pays only for the 42.8 MW it gets.
- Week 3: congestion is expensive. When the bottleneck is full, the LMP at both buses rises, and energy costs hundreds of millions of dollars per team instead of about $55M.

Here A has a plan that is best whatever B does: going large (3,653 beats 1,826 in Weeks 1–2, and 2,673 beats 1,772 or 1,336 in Week 3). This is a dominant strategy. B's best reply is large when A is small; when A is large, B gets the same 42.8 MW either way. So the game settles with A large and B served only the room A leaves. Three lessons carry over to the real game: the rule rewards the site that serves the most MW per MW of shared network, so small differences in location decide who gets the room; asking for more than you are allocated costs nothing, but does not get you more either; and your best plan depends on where the others build and how their buses share your bottlenecks.

11.2 Run controlled experiments

Change one thing at a time and record every run:

Run	Change from baseline	Served fraction	Preview score
baseline	1 site, bus 1001, 40 MW		
bigger	same bus, 60 MW		
moved	40 MW at another bus		
generator	baseline + 20 MW generator		

Ask AI

Write a Python script that takes a list of candidate plans (each a list of sites with `bus_id` and capacities), writes each plan to its own folder `runs/<name>/plan.csv` with this header: [paste header], runs this command for each folder: [paste the preview command], reads `team_scores[0]` from each output JSON, and prints a table of name, `served_load_mwh`, and `score`. Run candidates one at a time.

12 Troubleshooting

Message	What to do
`header must be exactly: ...`	Copy the header from section 8. Check for extra spaces, a byte-order mark, or a renamed column.
`submission must be UTF-8 CSV`	Save as "CSV UTF-8". Avoid saving from Excel in its native format.
`expected exactly 7 fields`	A row has a missing or extra comma.
`unknown scenario bus N`	Look up the bus in `bus.csv` of the block passed to `--artifact-dir`.
`duplicate site_id X`	Give every row its own `site_id`.
`one CSV may contain only one team_id`	Use your team ID on every row.
`... must be non-negative`	Use zero or a positive number.
`... must be a finite decimal`	Use a plain number: no units, no blanks, no text.
`generator capacity cannot exceed connection capacity`	Lower G to at most C, or raise C.
`total connection capacity cannot exceed ... MW per team`	Lower the C of your sites so they add up to at most x GW.
`storage energy must equal four times storage power`	Set $B = 4P$, or both to 0.
`CONFIGURATION ERROR: ...`	The command is wrong, not your file. For example, `--artifact-dir` must point to an existing scenario folder.
`scenario block validation_status is not passed`	The preview is pointing at an unreleased block. Use a block your instructor has released.
`no valid CSV submissions were found`	The preview folder is empty or every CSV failed; the lines above it give each file's reason. Run the checker first.
`connection_capacity_mw is 0, so ...`	Set $C > 0$, or drop G and P.
`empty site: every capacity is 0`	Remove the row.
`command not found: ercot-game-...`	Activate the virtual environment and rerun the install step in section 4.
`No module named 'highspy'`	Rerun the install step; the solver is a required dependency.

If you are stuck, ask an AI tool, and give it everything it needs:

Ask AI

I am running this command: [paste]. It printed: [paste the full output]. The first three lines of my CSV are: [paste]. What is wrong and how do I fix it?

13 Before you submit

- The header is exactly the seven columns, with one row per site.
- The same `team_id` is used on every row, and every `site_id` is unique.
- Every `bus_id` exists in the block you are submitting for.
- $G \le C$ at every site, all sites' C add up to at most x GW, and storage is either $(0, 0)$ or $B = 4P$.

- For Submission 2, every Submission 1 row is unchanged.
- The checker prints `VALID` for the exact file you will upload.
- You can explain why you chose each bus and capacity.
- The file name and upload location match The game, and you upload before the deadline in .

14 After each submission: reading your results

After every submission the instructor releases:

- (private) the score breakdown: IT value, energy cost, generator operating cost, and the capital costs charged in that submission;
- (public) the service fraction α of every team;
- (public) whether the site allocation was triggered;
- (public) an interactive grid map drawn on a schematic layout of the network. Each bus is colored by the total connection capacity that all teams placed there together, and each line by how heavily it was loaded. Hover over a bus or line to see the numbers. Only totals per bus are shown, so the map does not say who built what.

Read them in this order:

1. Was the allocation triggered? If not, the combined plan fit the grid and every site has $\alpha = 1$. If it was, compare your α (your served share of the capacity you requested) with the other teams'. A low α means your sites competed for the same lines as sites that serve more MW per MW of network (section 2.2).
2. Did the grid beat running off the grid? A MW served from the grid nets about \$36.5M, a MW run off the grid about \$24.8M (section 2).
3. Which term drives your score? Usually IT value against site cost; in Week 3, check whether congestion made energy expensive.
4. How does it compare with your preview? If much less of your load was served than in the preview, the shortfall came from other teams' sites, which your preview did not include.
5. Where is the grid crowded? On the map, look for busy buses and heavily loaded lines near your sites, and for regions that still look quiet.

Planning the next submission. After Submission 1, every team's connected load becomes base load. Your instructor releases the Submission 2 data, the representative data with that load added, so you can recompute hosting capacity and preview new sites against what is actually left (appendix A). After a week, everything resets, but remember that the other teams have learned from the results too.

15 Frequently asked questions

Is the preview my score?
No. It leaves batteries idle and ignores other teams (section 9). Use it to compare your own candidates. All those simplifications are for reducing computational costs. If you have enough computational power, you can write a full evaluator and test your plan.

Does Week 1 count?
No. Weeks 2 and 3 count, and both submissions in each of those weeks count (The game).

When do I pay for energy?
Only in Week 3, in both submissions, at the LMP. Weeks 1 and 2 have no energy cost.

Can I schedule my battery or sell power from my generator?
No. The evaluator dispatches both, and they can only serve your own load.

What happens to my Submission 1 sites in Submission 2?
They become part of the base load: already connected, and already scored for their ten-year value. Your Submission 2 file lists only new sites.

Can I resize or move a site in Submission 2?
No. You can only add new sites, generators, and batteries.

What happens if my file is invalid?
It scores 0 for that submission (The game). Other teams are evaluated without it. Run the checker on the exact file you upload.

Why is my α zero?
The combined plan did not fit the grid, and the most megawatts in total were served by giving the room your sites needed to other sites (section 2.2). You pay nothing for capacity you were not allocated. Move to buses with more room, or add a generator or battery so that your site needs less of the network.

Can other teams see my plan?
Not in detail. They can see every team's α, and the total connection capacity at each bus, not who built what.

My generator never seems to run. Is something wrong?
Probably not. At \$80/MWh or more it costs more than grid energy at about \$18/MWh, so the operator only uses it when the grid cannot deliver enough to your bus, or when congestion pushes the price at your bus above its cost.

16 Glossary

Term	Meaning	See
ACTIVSg2000	The synthetic 2,000-bus Texas grid used in the game.	section 1
Representative data	Five typical days of 120 hourly periods that stand for the next ten years.	section 7
Bus	A node of the network; every site connects to one bus.	section 1
Site	One row of your CSV: a bus plus the capacities you build there.	section 8
Connection capacity C	The load your site draws in every hour, in MW.	section 1
Submission 1 / 2	The two rounds of a week; Submission 1 load becomes Submission 2 base load.	section 1
Capital cost	One-time cost of building a site, generator, or battery.	section 2
IT value	\$1,000 earned for each MWh of your load that is served, over ten years.	section 2
DCOPF	DC optimal power flow, the dispatch the evaluator solves for each operating period.	section 3
LMP	Locational marginal price: the cost of one more MWh at a bus.	section 3
SOC	State of charge: the energy stored in a battery.	section 3
Allocation	How the evaluator shares the grid when not every request fits: each site gets a fraction α that maximizes the total served MW.	section 2.2
Service fraction α	The share of your requested capacity that is allocated (per site, or for your whole team).	section 2.2
Hosting capacity	The most new load a bus can take without making the grid infeasible.	section 7.1
Checker	`ercot-game-check-csv`; checks your file's format.	section 8
Preview	`ercot-game-preview`; an approximate score for your plan alone.	section 9
Evaluator	The instructor's official, exact scoring run with all teams.	section 1
Nash equilibrium	A set of plans in which no team gains by changing its own.	section 11.1

Appendices

A Hosting capacity in detail

The hosting-capacity idea comes from distribution planning, where it measures how much distributed generation a feeder can absorb before its performance becomes unacceptable [4]. The same idea applies to load. Recent work uses it to ask how large a data center a transmission bus can accept [23], and treats it as something that changes over time rather than a single number [6]. In this appendix, we state a version that matches this game.

A.1 Formulation

Write the per-period DCOPF in shift-factor (PTDF) form. Let $\mathbf{d}_t$ be the vector of bus loads in operating period t, $\mathbf{s}_t$ the fixed DC-line injections, M the matrix that maps each generator to its bus, Φ the PTDF matrix (the flow on each branch per MW injected at each bus and withdrawn at the reference bus), $\mathbf{F}$ the branch thermal limits, and $\mathbf{p}^{\min}, \mathbf{p}^{\max}$ the generator limits. The hosting capacity of bus k in period t is the optimal value of the linear program [1, 23]

$$
\begin{aligned}
\mathrm{HC}_{k,t} = \max_{L,\,\mathbf{p}^{\mathrm{g}},\,\mathbf{p},\,\mathbf{f}} \quad & L \\
\text{s.t.} \quad & \mathbf{p} = M\,\mathbf{p}^{\mathrm{g}} + \mathbf{s}_t - \mathbf{d}_t - L\,\mathbf{e}_k && \text{(net bus injections, new load at } k\text{)}, \\
& \mathbf{1}^{\top}\mathbf{p} = 0 && \text{(system balance)}, \\
& \mathbf{f} = \Phi\,\mathbf{p}, \quad |\mathbf{f}| \le \mathbf{F} && \text{(DC power flow, thermal limits)}, \\
& \mathbf{p}^{\min} \le \mathbf{p}^{\mathrm{g}} \le \mathbf{p}^{\max}, \quad L \ge 0 && \text{(generator limits)}.
\end{aligned}
$$

The decision variables are the new load L, the generator outputs $\mathbf{p}^{\mathrm{g}}$, the net bus injections $\mathbf{p}$, and the branch flows $\mathbf{f}$; $\mathbf{e}_k$ is the k-th unit vector. These are exactly the DCOPF constraints of appendix B with one extra load L at bus k, and the generation-cost objective is replaced by "make L as large as possible". (The PTDF form is equivalent to the voltage-angle form $f_\ell = b_\ell(\theta_{o(\ell)} - \theta_{r(\ell)})$ that many textbooks use.) At the optimum some branch is at its thermal limit, or the generators are at their limits: that binding constraint is the bottleneck for bus k.

Over the 120 representative hours. Your site's load is the same in every period, so the number that matters for a plan is

$$\mathrm{HC}_k = \min_t \mathrm{HC}_{k,t}.$$

This equals one large LP with a single L shared by every period. In practice you first compute $\mathrm{HC}_{k,t}$ for the hours with the highest system load. The minimum over a sample of periods is an upper bound on HC_k, so check more periods before relying on a number close to the limit.

Several sites at once: the hosting-capacity region. Replace $L\,\mathbf{e}_k$ by a vector $\mathbf{x} \ge 0$ of new loads, one per bus. The set of load vectors that keep every operating period feasible is

$$
\mathcal{H} = \left\{ \mathbf{x} \ge 0 : \begin{array}{c} \forall t \;\; \exists\, \mathbf{p}_t^{\mathrm{g}}, \mathbf{p}_t : \\ \mathbf{p}_t = M\,\mathbf{p}_t^{\mathrm{g}} + \mathbf{s}_t - \mathbf{d}_t - \mathbf{x}, \;\; \mathbf{1}^{\top}\mathbf{p}_t = 0, \\ |\Phi \mathbf{p}_t| \le \mathbf{F}, \;\; \mathbf{p}^{\min} \le \mathbf{p}_t^{\mathrm{g}} \le \mathbf{p}^{\max} \end{array} \right\}.
$$

It is a convex polytope, the projection of the DCOPF feasible set onto the new loads [9]. Each single-bus hosting capacity is where $\mathcal{H}$ meets one axis, so single-bus numbers do not add up. When two buses share a bottleneck, $\mathcal{H}$ is much smaller than the box formed by their individual hosting capacities: on the game grid at the peak hour, bus 1002 hosts 123 MW alone but only 81 MW with 60 MW at its neighbor 1010. The site

allocation in section 2.2 picks a point in $\mathcal{H}$ for all teams' sites, which is why your service depends on where everyone else builds.

With a private generator. A site at bus k with load L and a private generator $0 \le g_t \le G$ shows the grid only its net withdrawal $L - g_t$. The plan is feasible if and only if, in every period, some $g_t \in [0, G]$ gives $L - g_t \le \mathrm{HC}_{k,t}$, so

$$\mathrm{HC}_k^G = \min_t \mathrm{HC}_{k,t} + G.$$

This is exact when the generator is always available; outages, ramp limits, or fuel limits make it smaller. The generator runs only when the grid is short or when its cost is below the local price, and you pay for it through capital and an operating cost of at least \$80/MWh.

With storage: the periods become coupled. A battery at bus k with power P, energy B, signed output u_t (positive means discharging), and stored energy e_t gives the multi-period LP

$$\begin{aligned} \mathrm{HC}_k^S = \max_{L,\,u,\,e} \quad & L \\ \text{s.t.} \quad & L - u_t \le \mathrm{HC}_{k,t}, \quad -P \le u_t \le P && \forall t, \\ & e_t = e_{t-1} - \Delta t\, u_t, \quad 0 \le e_t \le B && \forall t, \\ & e_0 = e_T, \end{aligned}$$

where $e_0 = e_T$ is the daily cycle (in the game, $e_0 = e_T = B/2$ with T the periods of one day). The battery discharges in tight periods and recharges in slack ones [13]. Summing the energy equation over the cycle gives $\sum_t u_t = 0$; summing the first constraint then gives

$$\mathrm{HC}_k^S \le \frac{1}{T} \sum_t \mathrm{HC}_{k,t},$$

so storage lifts the limit at most from the minimum to the average headroom of the day. Its power ($L \le \mathrm{HC}_{k,t} + P$) and energy B keep it below that bound.

Further reading: flexible connection. In this game your load is fixed: the site draws its full capacity in every operating period, so a single tight period sets HC_k. Gu, Chen, and Qin [10] ask what happens if a new load instead agrees to be curtailed in a small number of time slots per year, an arrangement called flexible connection:

$$\max_{L,\,\delta} \; L \quad \text{s.t.} \quad L - \delta_t \le \mathrm{HC}_{k,t}, \quad \delta \ge 0, \quad \|\delta\|_0 \le K,$$

where δ_t is the load curtailed in period t and $\|\delta\|_0$ counts the curtailed periods, which makes the problem nonconvex. Its solution is

$$\mathrm{HC}_k^{(K)} = \mathrm{HC}_{k,[K+1]},$$

the $(K+1)$-th smallest of the operating-period hosting capacities: you are no longer limited by the K tightest periods. Their distribution-feeder case study finds that curtailing in only 1% of the 15-minute slots of a year (about 88 hours) raises the hosting capacity by about 78%, that the gain grows more slowly as K increases (see also [17]), and that most required curtailments are shallow. Letting the load delay its consumption instead of dropping it unlocks at most as much. A follow-up paper extends the approach to grids where only some buses are measured [11].

You cannot curtail in this game, but your own generator or battery can play the same role: whatever the grid cannot deliver in the K tightest periods must come from on-site resources, that is, $C - \mathrm{HC}_{k,t}$ MW in each of those periods. To see whether that is worth it at a bus you like, compute $\mathrm{HC}_{k,t}$ for the peak periods, sort them, and compare the smallest value with, say, the 25th smallest. The difference is the extra capacity that covering the 24 tightest periods would unlock, and the gap in each of those periods tells you the generator or battery power you would need.

How much should you trust a hosting-capacity number?

- Security. The game uses a plain DCOPF with no N-1 contingency or reserve constraints; dropping N-1 overstates data-center hosting capacity by about 20% on the IEEE RTS-24 system [23].
- Dynamics. The data center's own ride-through limits (UPS, converters) can be the binding constraint [16].
- Model. DC power flow has no voltage or reactive-power limits and no losses.
- Simultaneity. Single-bus numbers ignore everyone else who is connecting at the same time; that is what the region $\mathcal{H}$ and the site allocation are for.

A.2 Computing it

You do not need to write the LP yourself. Since the set of feasible L is an interval $[0, \mathrm{HC}_{k,t}]$, a bisection that asks "is the DCOPF still feasible with L extra MW at bus k?" finds the hosting capacity. The GitHub repository's solver answers that question directly; each check takes well under a second on a laptop. (`ercot-game-hosting-capacity` solves the LP itself and is faster for many buses.) Run from the GitHub repository root:

```
from tools.activsg2000.dcopf import DCOPFInfeasible, DCOPFUnresolved, solve_prepared_period

data = "data/processed/activsg2000_representative_v1"


def feasible(bus_id, extra_mw, period):
    """True if the grid can serve extra_mw at bus_id in this period."""
    try:
        solve_prepared_period(data, period_index=period,
                              additional_demand_mw_by_bus={bus_id: extra_mw})
        return True
    except (DCOPFInfeasible, DCOPFUnresolved):
        return False  # near the limit HiGHS may not resolve; count it as "no"


def hosting_capacity(bus_id, period, upper_mw=1000.0, tol_mw=1.0):
    """Largest extra load at bus_id that keeps this period feasible."""
    lo, hi = 0.0, upper_mw
    if feasible(bus_id, hi, period):
        return hi                       # at least upper_mw
    while hi - lo > tol_mw:             # feasible loads form an interval [0, HC]
        mid = (lo + hi) / 2
        if feasible(bus_id, mid, period):
            lo = mid
        else:
            hi = mid
    return lo


peak_period = 109  # highest system load: 1 p.m. on the peak-load day
for bus in ["1001", "1002"]:
    print(bus, [round(hosting_capacity(bus, period)) for period in (0, peak_period)])
```

Output:

```
1001 [391, 332]
1002 [129, 123]
```

To find the hours with the highest system load, add up the bus loads:

```
import pandas as pd

demand = pd.read_csv(f"{data}/demand_bus.csv", index_col=0)
peak_periods = list(demand.sum(axis=1).sort_values(ascending=False).index[:24])
peak_period = peak_periods[0]
print(peak_periods[:5])
```

To check how a neighbor affects you, pass both loads at once inside `feasible`:

```
additional_demand_mw_by_bus={bus_id: extra_mw, "1010": 60}
```

The solver only runs on data that your instructor has released and validated (section 9). For Submission 2, point `data` at the Submission 2 data, which already contains every team's Submission 1 load.

Ask AI

Here is my Python function `hosting_capacity(bus_id,period)`: [paste it]. Write a script that computes it for a list of candidate buses over the 24 peak-load periods in `peak_periods`, keeps the minimum for each bus, and prints a table sorted from largest to smallest. Use `concurrent.futures` to run buses in parallel.

B The evaluator in full

This appendix writes out the problem the evaluator solves for one submission, over the 120 representative hours.

Sets and data. Buses n and branches ℓ; the matrix M that maps existing generators g to their buses, with limits $\mathbf{p}^{\min}, \mathbf{p}^{\max}$ and marginal costs $\mathbf{c}$; the PTDF matrix Φ (the flow on each branch per MW injected at each bus and withdrawn at the reference bus) and branch thermal limits $\mathbf{F}$; base loads $\mathbf{d}_t$ and fixed DC-line injections $\mathbf{s}_t$ (vectors over buses); and hourly periods $t = 1, \ldots, T$ with $T = 120$. Team i has sites s at buses $k(s)$ with capacities C_s, G_s, P_s, B_s and site fraction α_s ($\alpha_s = 1$ unless the allocation of appendix C is needed; it scales C_s, G_s, P_s, B_s together); $\mathbf{e}_n$ is the unit vector of bus n.

Decision variables, for every operating period t. Existing generation $\mathbf{p}^{\mathrm{g}}_t$; net bus injections $\mathbf{p}_t$ and branch flows $\mathbf{f}_t$; private generation in three cost segments $q^m_{s,t}$, $m = 1, 2, 3$, with costs $c^1, c^2, c^3 = 80, 150, 250$ \$/MWh and widths $w^1, w^2, w^3 = 0.25, 0.5, 0.25$; battery net output $u_{s,t}$ in MW (positive means discharging) and state of charge $e_{s,d,h}$ in MWh. Each representative day d contains 24 hourly periods $h = 1, \ldots, 24$, and period $t = 24(d-1) + h$ is the h-th of day d; $e_{s,d,h}$ is the stored energy at the end of that period and $e_{s,d,0}$ is the stored energy at the start of day d. The period length is $\Delta t = 1$ hour.

Problem. The evaluator minimizes the total generation cost over the 120 hours,

$$\min \sum_{t=1}^{T} \left(\mathbf{c}^\top \mathbf{p}^{\mathrm{g}}_t + \sum_{s} \sum_{m=1}^{3} c^m q^m_{s,t} \right)$$

(multiplying this objective by Δt would not change the dispatch), subject to, for every period $t = 24(d-1)+h$:

$$\begin{aligned}
&\mathbf{p}_t = M\,\mathbf{p}^{\mathrm{g}}_t + \sum_s \mathbf{e}_{k(s)}\Big(\sum_m q^m_{s,t} + u_{s,t} - \alpha_s C_s\Big) + \mathbf{s}_t - \mathbf{d}_t && \text{(net injections)},\\
&\mathbf{1}^\top \mathbf{p}_t = 0 && \text{(system balance)},\\
&\mathbf{f}_t = \Phi\,\mathbf{p}_t, \quad |\mathbf{f}_t| \le \mathbf{F} && \text{(DC flow, limits)},\\
&\mathbf{p}^{\min} \le \mathbf{p}^{\mathrm{g}}_t \le \mathbf{p}^{\max}, \qquad 0 \le q^m_{s,t} \le w^m \alpha_s G_s && \forall s, m,\\
&e_{s,d,h} = e_{s,d,h-1} - \Delta t\, u_{s,t}, \quad 0 \le e_{s,d,h} \le \alpha_s B_s && \forall s,\\
&|u_{s,t}| \le \alpha_s P_s, \quad \Delta t\, u_{s,t} \le e_{s,d,h-1} && \forall s,\\
&e_{s,d,0} = 0.5\,\alpha_s B_s, \qquad e_{s,d,24} = 0.5\,\alpha_s B_s && \forall s, d,\\
&\sum_{s\in\text{team } i,\ k(s)=n} u_{s,t} \le \sum_{s\in\text{team } i,\ k(s)=n} \alpha_s C_s && \forall i, n,
\end{aligned}$$

where the last constraint keeps a team's batteries from discharging more than that team's load at a bus (a team's generators are limited the same way). A generator may run in every hour. Batteries couple the 24 hours within each representative day, but not different days. The evaluator therefore solves one independent linear program per day; each day starts and ends at half charge. How it solves these daily problems quickly is the subject of appendix B.1. The grid operator (instructor) solves it exactly. The preview drops storage, which makes every operating period independent (section 9).

Prices and score. The LMP $\lambda_{n,t}$ is the cost of serving one more MW of load at bus n in period t. In the PTDF form it splits into an energy part and a congestion part,

$$\boldsymbol{\lambda}_t = \pi_t\,\mathbf{1} - \Phi^\top \boldsymbol{\mu}_t,$$

where π_t is the dual variable of the system balance and $\boldsymbol{\mu}_t = \boldsymbol{\mu}^+_t - \boldsymbol{\mu}^-_t$ collects the dual variables of the upper and lower flow limits. Without congestion ($\boldsymbol{\mu}_t = 0$) every bus has the same price π_t. When a line is at its limit, load at a bus that adds to the flow on that line ($\Phi_{\ell n} < 0$) pays more, and load at a bus that relieves it pays less. Prices are clipped at zero. Team i's grid purchase at bus n is the part of its load that its own generation and batteries do not cover, $\pi_{i,n,t} = \max\{0,\ \sum_s(\alpha_s C_s - \sum_m q^m_{s,t} - u_{s,t})\}$, summed over its sites at n. With the ten-year period weights $w_t = \Delta t \times d_t \times A$ of section 1 (d_t the days per year that period t's day stands for, $A = 7.02$),

$$\begin{aligned}
\text{IT value}_i &= 1{,}000 \sum_t w_t \sum_{s\in i} \alpha_s C_s,\\
\text{energy cost}_i &= \sum_t w_t \sum_n \lambda_{n,t}\,\pi_{i,n,t},\\
\text{gen. operating}_i &= \sum_t w_t \sum_{s\in i}\sum_m c^m q^m_{s,t},\\
\text{capital}_i &= \sum_{s\in i,\ \text{new}} \alpha_s\big(25\text{M}\,C_s + 2.0\text{M}\,G_s\\
&\qquad + 0.4\text{M}\,P_s + 0.25\text{M}\,B_s\big),
\end{aligned}$$

where the energy cost applies in Week 3 only, and "new" means built in this submission. Capital is not discounted. In Submission 2, the base load $\mathbf{d}_t$ also contains the hourly net grid withdrawal of every Submission 1 site, taken from the Submission 1 dispatch.

B.1 Solving a day quickly: Benders decomposition

A day on the full network is a large linear program: 24 copies of the 2,000-bus DCOPF, about 125,000 constraints, tied together only by a few devices at the teams' sites. Solving it in one piece is slow. When the

plans fit, the evaluator therefore splits each day into a small problem about the batteries and 24 ordinary hourly DCOPFs. This is Benders decomposition [2].

Split the day. Collect the battery outputs of the day, $u_{s,h}$, in a vector x, and write X for the set allowed by the daily constraints above (state of charge, power limits, and "serve only your own load"). Once x is fixed, each hour is a DCOPF in which the devices are fixed injections at their buses:

$$\phi_h(x) = \min_{\mathbf{p}^{\mathrm{g}}_h} \ \mathbf{c}^\top \mathbf{p}^{\mathrm{g}}_h \quad \text{s.t. the network constraints of hour } h,$$

the cost of running the grid in hour h (with a large penalty on any load that cannot be served, so ϕ_h is always defined). The day's problem is then

$$\min_{x \in X} \ \sum_{h=1}^{24} \phi_h(x)$$

(the generators stay in the hourly DCOPFs, priced by their fuel curve). Each ϕ_h is convex and piecewise linear in x, but we do not have a formula for it, only the ability to evaluate it by solving one hourly DCOPF.

Learn ϕ_h from cuts. Solving the DCOPF of hour h at a trial point $\hat{x}$ gives its cost $\phi_h(\hat{x})$ and the LMPs at the site buses. The LMPs say how the cost changes if a device injects one more MW, so they give a linear lower bound, a cut, that holds for every x:

$$\phi_h(x) \ \geq \ \phi_h(\hat{x}) - \boldsymbol{\lambda}_h(\hat{x})^\top (x_h - \hat{x}_h),$$

where x_h are the device injections in hour h. The small master problem replaces each ϕ_h by a variable θ_h that must lie above all cuts found so far:

$$\min_{x \in X,\ \theta} \ \sum_h \theta_h \quad \text{s.t.} \quad \theta_h \geq \text{every cut of hour } h.$$

The algorithm alternates:

1. Solve the master problem. Its value is a lower bound on the day's cost, and its solution is the next trial point $\hat{x}$.
2. Solve the 24 hourly DCOPFs at $\hat{x}$ (independently, in parallel). Their total cost is an upper bound: it is the cost of an actual dispatch.
3. Add one new cut per hour to the master and repeat until the two bounds agree (to a relative gap of 10^{-7}).

Because every ϕ_h is piecewise linear, the loop ends after finitely many cuts at the exact optimum of the daily LP; on the game's package it typically needs about 20 iterations. The master is small (it has only the batteries' variables), and each hourly DCOPF is the same size as a hosting-capacity LP, so the whole day solves in seconds instead of the many minutes the one-piece LP can take. The LMPs of the full day are recovered from the master's cut multipliers.

Why this matters for you

You do not need Benders decomposition to plan: hosting capacity, generators and batteries can be reasoned about bus by bus as in appendix A. But the idea is worth knowing. "Fix the few coupling decisions, solve many small independent problems, and feed their prices back as cuts" is how large planning and scheduling problems are solved in practice.

C The site allocation in detail

When the combined plan of all valid teams is infeasible, the evaluator scales the sites down. Every site s (of any team) gets its own fraction $\alpha_s \in [0, 1]$. ERCOT does not publish a formula for splitting capacity among the loads it studies together [8]; the game uses the rule below so that every team can reproduce it.

C.1 What a fraction does

A fraction scales the whole site

Site s with fraction α_s is treated exactly as if you had submitted

$$\alpha_s C_s, \quad \alpha_s G_s, \quad \alpha_s P_s, \quad \alpha_s B_s :$$

its load is $\alpha_s C_s$ in every period, its generator may produce up to $\alpha_s G_s$, and its battery is $\alpha_s P_s$ MW and $\alpha_s B_s$ MWh, starting and ending each day at $\alpha_s B_s/2$. Capital is charged on the scaled capacities. Different sites of the same team can get different fractions.

C.2 The rule

Stage 1: serve the most. The evaluator finds the largest total served capacity

$$V^\star = \max_{\boldsymbol{\alpha} \in [0,1]^S} \sum_s \alpha_s C_s \quad \text{s.t. the constraints of appendix B hold in all 120 periods,}$$

with α_s in place of $\alpha_{i(s)}$ and the generator and battery limits scaled as above. This is a linear program: the fractions enter every constraint linearly.

Stage 2: break ties evenly. Usually many fraction vectors serve $V^\star$ (for example, two identical sites at the same bus can split the room in any way). Among them the evaluator picks

$$\boldsymbol{\alpha}^\star = \arg\min_{\boldsymbol{\alpha}} \sum_s C_s (1 - \alpha_s)^2 \quad \text{s.t.} \quad \sum_s \alpha_s C_s = V^\star \text{ and the constraints above.}$$

The objective is strictly convex in $\boldsymbol{\alpha}$ and the feasible set is convex, so $\boldsymbol{\alpha}^\star$ is unique: the allocation does not depend on the solver or on the order of the files. Identical sites at the same bus get the same fraction.

C.3 What the rule rewards

- MW served per MW of network. Stage 1 counts every served MW the same, wherever it is. A site that needs less of the scarce network for each MW it serves wins the room. Example: a bus with 70 MW of room in every hour, site A asks for 60 MW with no generator and site B for 60 MW with a 30 MW generator. Serving B in full uses only 30 MW of the room, so the rule serves B in full and A at $40/60 = 67\%$: 100 MW in total, more than any even split.
- Off-grid sites are never cut. A site with $G = C$ needs no network, so it always gets $\alpha_s = 1$. It earns less per MW than a site served from the grid (section 2), which is why it is the reference plan.
- Location matters through the bottlenecks. When a line limits several buses, a MW at a bus with a small shift factor on that line uses less of it, so sites there are served first.
- No minimum allocation. A site with a small fraction is still served that fraction, and pays only for it.

C.4 How it is computed

The fractions are the same in all 120 periods, so they couple all five days; within a day, batteries couple the hours. The evaluator solves the two stages with a cutting-plane method that never builds the 120-period problem in one piece:

1. A small master problem over the fractions only (a linear program in stage 1, a quadratic program in stage 2) proposes $\hat{\boldsymbol{\alpha}}$.

2. For each representative day, a load-shedding LP measures how far $\hat{\boldsymbol{\alpha}}$ is from fitting: it minimizes the total load that would have to be shed in that day's 24 hours. The shed is a convex, piecewise-linear function of $\boldsymbol{\alpha}$, and the LP's reduced costs give its slope.
3. If a day sheds load, its value and slope give a linear cut $\text{shed}_d(\hat{\boldsymbol{\alpha}}) + \mathbf{g}_d^\top(\boldsymbol{\alpha} - \hat{\boldsymbol{\alpha}}) \le 0$ that every fitting $\boldsymbol{\alpha}$ satisfies but $\hat{\boldsymbol{\alpha}}$ does not. It is added to the master, and the loop repeats until no day sheds load.

To avoid jumping between extreme proposals, each proposal is first tested halfway towards the last fractions known to fit (*in-out* separation); the five days are independent and are solved in parallel. The final fractions are then dispatched with one exact LP per day (appendix B). If even the base network cannot be dispatched, the evaluator falls back to leaving one team out at a time, then each team alone, and records what happened.

D Where the numbers come from

The numbers are instructional, informed by public sources and rounded. They are not tariffs, forecasts, or ERCOT rules.

Quantity	Value	Basis
IT value	\$1,000/MWh	An instructor-chosen net margin per MWh for an AI data center, between colocation rent (about \$2.4M per MW per year, or \$270/MWh [3]) and projected GPU-cloud revenue (about \$15M per MW per year, or \$1,700/MWh [7]).
Planning horizon	10 years at 7%	A data center runs for about ten years; per-MWh terms are discounted with the annuity factor $A = \sum_{y=1}^{10} 1.07^{-y} = 7.02$. The 7% rate is the one OMB Circular A-4 (2003) recommended to approximate the average return on private capital [21]; the 2023 revision lowered the default to 2% [22], but private investors in data centers expect higher returns, so 7% is a reasonable instructional choice.
Representative days	5 days, 120 hours	Four medoid days from clustering the daily load profiles of one year of hourly ACTIVSg2000-based data, plus the peak-load day; weights sum to 365.
Site cost	\$25M/MW	Cost of building an AI data center per MW of IT load: about \$10.7M/MW for the building and up to \$25M/MW with AI fit-out [12], or \$21–31M/MW including servers [14].
Generator capital	\$2.0M/MW	Installed cost of on-site gas engines and aeroderivative turbines for data centers, about \$1,700–2,000/kW [15]; EIA reference costs are in the same range [19].
Generator cap	$G \le C$, any hours	A site may carry its whole load on its own generator, which may run in every hour; with $G = C$ it can run off the grid. This makes off-grid operation the reference plan for the absolute points.
Team cap	x GW per submission	An instructional limit on each team's total connection capacity, so that scores stay bounded and comparable with the reference plan. x will be set once we know how many teams take part.
Generator operating cost	\$80/150/250 per MWh	A rising cost curve for on-site gas generation with a diesel-backup premium; the exact shape is instructional [20].
Storage capital	\$400k/MW + \$250k/MWh	Rounded from the 2028 moderate projection for four-hour batteries [5]; \$1.4M per MW of four-hour storage.
Grid energy price	LMP from the dispatch	The bus-level dual price of the DCOPF; about \$17–20/MWh before students add load.
Network	ACTIVSg2000	Public synthetic Texas grid [18]; the load profiles are synthetic, not historical data.

E Working as a team

How to collaborate: use a private Git repository. Commit your CSVs, scripts, and run log so everyone works from the same files and you can always return to the exact plan you submitted. Keep the repository private: your plan is your competitive information.

```
# illustrative: create a private repository on GitHub first, then
git clone git@github.com:<you>/team-blue.git && cd team-blue
git add runs/ scripts/ runlog.csv
git commit -m "Week 2 Submission 1: plan-a, 2 sites"
git push      # to a private repository shared only with your team
```

Keep one candidate per folder. The preview dispatches every CSV in a folder together, so give each candidate its own folder:

```
runs/
  week2-sub1/plan-a/plan.csv
  week2-sub1/plan-a.json        <- preview output
  week2-sub1/plan-b/plan.csv
runlog.csv
```

Large preview outputs need not be committed; the CSV and the command are enough to recreate them.

Log every run. One row per run is enough:

Date	Block	Candidate	Change	Served	Score	Note
10-20	2028 jan_jun	plan-a	baseline	97%	...	
10-20	2028 jan_jun	plan-b	site 2: 40→60 MW	81%	...	below break-even

Split the work. For example, one person screens buses and computes hosting capacity, one writes and checks CSVs, and one runs previews and keeps the log. Review the final CSV together before you submit.

References

[1] Mansoor Alturki, Amin Khodaei, Aleksi Paaso, and Shay Bahramirad. Optimization-based distribution grid hosting capacity calculations. *Applied Energy*, 219:350–360, 2018.

[2] J. F. Benders. Partitioning procedures for solving mixed-variables programming problems. *Numerische Mathematik*, 4(1):238–252, 1962.

[3] Bitdeer Technologies Group. Form 6-k, exhibit 99.1. U.S. Securities and Exchange Commission, 2026.

[4] Math H. J. Bollen and Fainan Hassan. *Integration of Distributed Generation in the Power System*. Wiley-IEEE Press, Hoboken, NJ, 2011.

[5] Wesley Cole and Abhishek Karmakar. Cost projections for utility-scale battery storage: 2023 update. Technical report, National Renewable Energy Laboratory, 2023. The 2028 moderate four-hour projection is 351 dollars per kWh in 2022 dollars.

[6] Tiago E. C. de Oliveira, Math H. J. Bollen, Paulo F. Ribeiro, Pedro M. S. de Carvalho, Antônio C. Zambroni, and Benedito D. Bonatto. The concept of dynamic hosting capacity for distributed energy resources: Analytics and practical considerations. *Energies*, 12(13):2576, 2019.

[7] Digi Power X Inc. Form 8-k, exhibit 99.1. U.S. Securities and Exchange Commission, 2026.

[8] ERCOT. Trending topics: ERCOT's new batch connection process for large electricity users, June 2026.

[9] Sicheng Gong and J. F. G. Cobben. Adaptive facet selection in multidimensional hosting capacity region assessment. *Electric Power Systems Research*, 234:110654, 2024.

[10] Nan Gu, Ge Chen, and Junjie Qin. The role of flexible connection in accelerating load interconnection in distribution networks. In *Proceedings of the Hawaii International Conference on System Sciences (HICSS)*, 2026. arXiv:2510.11476.

[11] Nan Gu, Vassilis Kekatos, and Junjie Qin. Flexible connection to accelerate load interconnection in partially measured distribution grids. In *Proceedings of the 17th ACM International Conference on Future and Sustainable Energy Systems (e-Energy)*, 2026.

[12] Jones Lang LaSalle IP, Inc. 2026 global data center outlook, 2026. Data-center shell/core and AI fit-out construction-cost context.

[13] L. H. Macedo, J. F. Franco, R. Romero, M. A. Ortega-Vazquez, and M. J. Rider. Increasing the hosting capacity for renewable energy in distribution networks. In *IEEE PES Innovative Smart Grid Technologies Conference (ISGT)*, 2017.

[14] Morgan Stanley Research. Bridging the data center gap, 2025. AI data-center construction-cost range including GPUs and servers.

[15] Rabobank. The sprint: Data centers are building a parallel energy system in the US, 2026.

[16] Pengyu Ren, Wei Sun, and Fei Teng. Hosting capacity assessment of data centers with voltage ride-through capability in power systems. arXiv:2609.03030, 2026.

[17] Gobinda Chandra Sarker and Nathan Dahlin. Risk-aware hosting capacity analysis for flexible load interconnection in distribution networks. arXiv:2604.20185, 2026.

[18] Texas A&M University. Activsg2000 synthetic test case, 2019. Public synthetic transmission-network model; consult the repository's released scenario package for this tutorial.

[19] U.S. Energy Information Administration. Capital cost and performance characteristics for utility-scale electric power generating technologies, 2024. Technology-cost reference for the generator capital anchor.

[20] U.S. Energy Information Administration. How much electricity is generated by natural gas-fired power plants?, 2025. Heat-rate context for the instructional team-generator curve.

[21] U.S. Office of Management and Budget. Circular A-4: Regulatory analysis, 2003. Recommends discount rates of 3% and 7%; 7% approximates the average before-tax return on private capital.

[22] U.S. Office of Management and Budget. Circular A-4: Regulatory analysis (revised). 88 Fed. Reg. 77615, 2023. Lowers the default rate to 2% and drops the 7% rate.

[23] Haoxiang Wan, Linhan Fang, Jonghwan Kwon, Jesus Silva-Rodriguez, Jianyi Yang, Jaya Yellajosula, Fan Jiang, and Xingpeng Li. Impact of security modeling fidelity on hyperscale data center hosting capacity assessment. arXiv:2610.18405, 2026.

[24] Ray D. Zimmerman and Daniel Murillo-Sánchez. *MATPOWER User's Manual*, 8.1 edition, 2024. Case-format and power-flow conventions; accessed 2026-10-11.